

Part B: Payment Service Module (Candidate)

🕒 Audience	Candidate
≡ Description	The buggy payment service module in Python, Go, and Ruby. Export as PDF for candidates.
🕒 Section	Part B

Payment Idempotency Service

You are looking at a simplified backend service module from a fintech company. This service handles payment disbursement requests and ensures idempotency (duplicate requests return the same result rather than processing twice).

The service:

- Receives a disbursement request with a payment amount, recipient bank details, and an idempotency key
- Checks Redis cache to detect duplicate requests
- Validates the payment record's state before processing
- Calls an external payment provider API to execute the disbursement
- Records the result back to the database

Take your time to read the code. Use whatever tools you normally use. Then tell us:

1. What issues do you see?
2. Which one would you fix first, and why?
3. Fix it.

Python

```

import json
import redis
import requests
from dataclasses import dataclass
from typing import Optional

redis_client = redis.Redis(host='localhost', port=6379, db=0)

IDEMPOTENCY_EXPIRY = 600 # 10 minutes

# Module-level cache for the last processed key
_last_cache_key = None

@dataclass
class DisbursementResult:
    id: str
    status: str
    amount: float

def process_disbursement(payment_id: str, amount: float, bank
_code: str,
                        account_no: str, account_name: str,
                        idempotency_key: Optional[str] = None,
                        use_replica: bool = True) -> DisbursementResult:
    global _last_cache_key

    # Check idempotency cache
    if idempotency_key:
        cache_key = f"disbursement:{idempotency_key}"
        _last_cache_key = cache_key
        cached = redis_client.get(cache_key)

```

```

        if cached:
            data = json.loads(cached)
            return DisbursementResult(**data)

    # Look up the payment record
    payment = get_payment_record(payment_id, use_replica=use_replica)

    if payment is None:
        raise ValueError(f"Payment {payment_id} not found")

    # Validate state
    if payment['status'] not in ('compliance_passed', 'pending_disbursement'):
        raise ValueError(f"Payment {payment_id} in invalid state: {payment['status']}")

    # Call external payment provider
    response = requests.post(
        'https://api.payment-provider.com/v1/disbursements',
        json={
            'amount': amount,
            'reference_id': payment_id,
            'bank_code': bank_code,
            'account_number': account_no,
            'account_name': account_name,
        },
        timeout=30
    )
    result = response.json()

    if result.get('errors'):
        payment['status'] = 'failed'
        save_payment_record(payment)
        raise RuntimeError(f"Disbursement failed: {result['errors']}")

```

```

# Update payment record
payment['status'] = 'disbursement_processing'
payment['provider_reference'] = result['data']['id']
save_payment_record(payment)

disbursement = DisbursementResult(
    id=result['data']['id'],
    status=result['data']['status'],
    amount=amount
)

# Cache the result for idempotency
if _last_cache_key:
    redis_client.setex(
        _last_cache_key,
        IDEMPOTENCY_EXPIRY,
        json.dumps({'id': disbursement.id, 'status': disbursement.status,
                    'amount': disbursement.amount}))

return disbursement

def get_payment_record(payment_id: str, use_replica: bool = True) -> Optional[dict]:
    """Fetch payment from database. Uses read replica by default."""
    if use_replica:
        # Read from replica for performance
        return db_replica.query(f"SELECT * FROM payments WHERE id = '{payment_id}'")
    return db_primary.query(f"SELECT * FROM payments WHERE id = '{payment_id}'")

```

```
def save_payment_record(payment: dict) -> None:
    """Save payment to primary database."""
    db_primary.save('payments', payment)
```

Go

```
package disbursement

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "bytes"
    "time"

    "github.com/redis/go-redis/v9"
)

const idempotencyExpiry = 10 * time.Minute

// Module-level cache for the last processed key
var lastCacheKey string

var redisClient *redis.Client

type DisbursementResult struct {
    ID      string `json:"id"`
    Status  string `json:"status"`
    Amount  float64 `json:"amount"`
}

type DisbursementRequest struct {
    PaymentID string
```

```

    Amount          float64
    BankCode         string
    AccountNo        string
    AccountName       string
    IdempotencyKey    string
    UseReplica        bool
}

func ProcessDisbursement(ctx context.Context, req DisbursementRequest) (*DisbursementResult, error) {
    // Check idempotency cache
    if req.IdempotencyKey != "" {
        cacheKey := fmt.Sprintf("disbursement:%s", req.IdempotencyKey)
        lastCacheKey = cacheKey
        cached, err := redisClient.Get(ctx, cacheKey).Result()

        if err == nil {
            var result DisbursementResult
            json.Unmarshal([]byte(cached), &result)
            return &result, nil
        }
    }

    // Look up the payment record
    payment, err := getPaymentRecord(ctx, req.PaymentID, req.UseReplica)
    if err != nil {
        return nil, fmt.Errorf("payment %s not found: %w", req.PaymentID, err)
    }

    // Validate state
    if payment.Status != "compliance_passed" && payment.Status != "pending_disbursement" {
        return nil, fmt.Errorf("payment %s in invalid state: %s", req.PaymentID, payment.Status)
    }
}

```

```

%s", req.PaymentID, payment.Status)
}

// Call external payment provider
payload, _ := json.Marshal(map[string]interface{}{
    "amount":      req.Amount,
    "reference_id": req.PaymentID,
    "bank_code":    req.BankCode,
    "account_number": req.AccountNo,
    "account_name": req.AccountName,
})

httpClient := &http.Client{Timeout: 30 * time.Second}
resp, err := httpClient.Post(
    "https://api.payment-provider.com/v1/disbursements",
    "application/json",
    bytes.NewBuffer(payload),
)
if err != nil {
    return nil, fmt.Errorf("provider request failed: %w",
err)
}
defer resp.Body.Close()

var apiResp map[string]interface{}
json.NewDecoder(resp.Body).Decode(&apiResp)

if _, hasErrors := apiResp["errors"]; hasErrors {
    payment.Status = "failed"
    savePaymentRecord(ctx, payment)
    return nil, fmt.Errorf("disbursement failed: %v", api
Resp["errors"])
}

// Update payment record
data := apiResp["data"].(map[string]interface{})

```

```

payment.Status = "disbursement_processing"
payment.ProviderReference = data["id"].(string)
savePaymentRecord(ctx, payment)

result := &DisbursementResult{
    ID:      data["id"].(string),
    Status:  data["status"].(string),
    Amount:  req.Amount,
}

// Cache the result for idempotency
if lastCacheKey != "" {
    cacheData, _ := json.Marshal(result)
    redisClient.SetEx(ctx, lastCacheKey, string(cacheData), idempotencyExpiry)
}

return result, nil
}

func getPaymentRecord(ctx context.Context, paymentID string,
useReplica bool) (*Payment, error) {
    if useReplica {
        return dbReplica.Query(ctx, "SELECT * FROM payments WHERE id = '"+paymentID+"'")
    }
    return dbPrimary.Query(ctx, "SELECT * FROM payments WHERE id = '"+paymentID+"'")
}

func savePaymentRecord(ctx context.Context, payment *Payment)
error {
    return dbPrimary.Save(ctx, "payments", payment)
}

```


Ruby

```
module Disbursement
  IDEMPOTENCY_EXPIRY = 600 # 10 minutes

  # Module-level cache for the last processed key
  @last_cache_key = nil

  class << self
    attr_accessor :last_cache_key
  end

  def self.process_disbursement(payment_id:, amount:, bank_code:,
                                account_no:, account_name:,
                                idempotency_key: nil, use_replica: true)
    # Check idempotency cache
    if idempotency_key
      cache_key = "disbursement:#{idempotency_key}"
      self.last_cache_key = cache_key
      cached = RedisConnector.instance.redis.get(cache_key)
      if cached
        data = JSON.parse(cached, symbolize_names: true)
        return OpenStruct.new(data)
      end
    end

    # Look up the payment record
    payment = get_payment_record(payment_id, use_replica: use_replica)
    raise "Payment #{payment_id} not found" if payment.nil?

    # Validate state
    unless %w[compliance_passed pending_disbursement].include

```

```

e?(payment.status)
  raise "Payment #{payment_id} in invalid state: #{payment.status}"
end

# Call external payment provider
response = HTTParty.post(
  'https://api.payment-provider.com/v1/disbursements',
  body: {
    amount: amount,
    reference_id: payment_id,
    bank_code: bank_code,
    account_number: account_no,
    account_name: account_name
  }.to_json,
  headers: { 'Content-Type' => 'application/json' },
  timeout: 30
)
result = JSON.parse(response.body, symbolize_names: true)

if result[:errors]
  payment.update!(status: 'failed')
  raise "Disbursement failed: #{result[:errors]}"
end

# Update payment record
payment.update!(
  status: 'disbursement_processing',
  provider_reference: result[:data][:id]
)

disbursement = OpenStruct.new(
  id: result[:data][:id],
  status: result[:data][:status],
  amount: amount
)

```

```

# Cache the result for idempotency
if self.last_cache_key
  RedisConnector.instance.redis.setex(
    self.last_cache_key,
    IDEMPOTENCY_EXPIRY,
    { id: disbursement.id, status: disbursement.status,
      amount: disbursement.amount }.to_json
  )
end

disbursement
end

def self.get_payment_record(payment_id, use_replica: true)
  if use_replica
    ReplicaHelper.on_replica { Payment.find_by(id: payment_
id) }
  else
    Payment.find_by(id: payment_id)
  end
end
end
end

```

Context for the Candidate

This service runs in a fintech environment where:

- Payments involve real money. A duplicate disbursement means a user gets paid twice.
- The external payment provider charges per API call and has rate limits.
- The system handles ~500 requests/second at peak.
- There are separate read replicas and a primary database. Replicas may lag the primary by up to 2 seconds.

- Redis is used for caching and background job queues.

You have 40 minutes. Read the code, identify what concerns you, and fix the issue you consider most critical. Explain your reasoning as you go.